■ EduMentor AI – Complete Project Viva Master Guide

Full Project Architecture, Every Tech Stack Breakdown, Workflow & Top 20 Viva Questions

Language: English + Simple Tanglish Explanations with Real-World Analogies for Instant Viva Recall

■ 1. 1-MINUTE ELEVATOR PITCH FOR VIVA (EXAMINER-KU ENNA SOLLA VENDIYATHU?)

How to Start Your Viva Answer (The Killer Pitch)

Tanglish Meaning: *Sir/Madam, namma project* ***EduMentor AI*** *oru Automated Daily Knowledge Evaluation and Mentorship Analytics System. Traditional colleges-la faculty nadathuna topic student-ku purinjitha illayanu semester exam-la than theriya varum. EduMentor AI-la mentor daily oru topic assign pannuvanga (e.g. TCP Protocol). System udane Google Gemini AI moolama dynamic questions (MCQ, True/False, Fill in blanks) generate pannum with a timer. Students test attend panna udane, marks instant-aa evaluate aagi, weak students (<50% or 'didn't understand') identify aavanga. Deadline mudiyumbothu, background scheduler automatically faculty-ku complete attendance, marks list and reteach suggestions-oda email anuppidum! Koodave students-ku 24/7 AI tutor chat support irukku!*

Technical Explanation: In English (Examiner Pitch): EduMentor AI is an end-to-end intelligent pedagogical ecosystem bridging the feedback gap between students and educators. It integrates dynamic AI-powered daily micro-assessments (Google Gemini), real-time automated grading, background task scheduling (APScheduler), automated faculty intervention alerting (Resend/SMTP), and an interactive 24/7 AI doubt-clearing tutor built using React 19, FastAPI, and SQLAlchemy.

Real-life Example: **Analogy: Namma college-la doctor maathiri daily health checkup eduthu faculty-ku report kudukra AI!**

■■ 2. COMPLETE TECH STACK MASTER TABLE (ELLAMEY ETHUKKU USE PANDROM?)

Technology	What is it? (Definition)	Namma Project-la Ethukku?	Simple Real-Life Example
React 19	Component-based Frontend JavaScript Library for interactive Single Page Applications (SPA).	*Student Dashboard, Mentor Portal, Live Timer, Quiz Cards, and Real-time Chat UI build panna.*	Lego Blocks maathiri: Button, Navbar, Timer ellam re-usable pieces.
Vite 8	Next-generation lightning-fast frontend dev server & module bundler using native ES modules.	*React code-ah ultra-fast reload (Hot Module Replacement) panni production bundle create panna.*	Old Webpack cycle maathiri wait panna theva illa; Superfast Sports car maathiri 200ms-la run aagum.
Tailwind CSS v4	Utility-first CSS framework for rapid modern responsive UI styling without writing custom CSS files.	*Glassmorphic cards, gradients, modern dark/light styling, and responsive layout design panna.*	Ready-made clothes maathiri: `bg-blue-600 rounded-xl p-4` nu direct-aa HTML tag-laye style pannalam.
Framer Motion	Production-ready declarative animation library for React components.	*Quiz card flip, modal popups, score progress bar smooth animation, and page transitions-ku.*	PowerPoint slide animations maathiri UI elements smooth-aa paranthu vanthu settle aagum.
React Router v7	Client-side routing engine for multi-view Single Page Applications without browser reload.	*`/student/dashboard`, `/mentor/dashboard`, `/chat` nu seamless page navigation manage panna.*	Page refresh aagama TV channel switch pandra maathiri instant screen switch.

FastAPI	Modern, high-performance asynchronous Python web framework based on OpenAPI & Starlette.	*Namma REST API Backend engine: Auth, Timetable, Daily Topic, Scoring, Gemini Quiz Generation handle panna.*	Express counter maathiri: Request vantha async-aa fast-aa response kudukkum, Swagger docs automatic-aa generate pannum.
Uvicorn	Lightning-fast ASGI (Asynchronous Server Gateway Interface) web server implementation for Python.	*FastAPI application-ah run panni incoming HTTP/HTTPS browser requests-ah handle panna.*	Engine of a Train: FastAPI is the train coaches, Uvicorn is the high-power electric engine driving it.
Pydantic v2	Data validation and parsing library using standard Python type hints (`schemas.py`).	*Frontend-la irunthu vara student answers, login credentials data types accurate-aa validate panna.*	Airport Security Scanner: Wrong data type vantha ulla vidaama error throw pannidum.
SQLAlchemy 2.0	Object Relational Mapper (ORM) translating Python class objects into Database SQL queries.	*Direct SQL query ezhuthama `StudentScore`, `TopicHistory`, `QuizAttempt` Python models moolama DB operate panna.*	Translator: Namma Python-la pesina, athu Database SQL language-ku translate pannikkum.
PostgreSQL / SQLite	ACID-compliant Relational Database Management System (RDBMS) storing relational tables.	*Students data, login passwords, quiz marks, chat history, topic assignments safely store panna.*	College Central Record Room: Ella register files-um orderly racks-la save panni vachirukkum.
Google Gemini AI	Google's Multimodal Generative Large Language Model (`gemini-2.0-flash` & `1.5-flash`).	*Topic-ku relevant MCQ/TF questions automatically generate panna + 24/7 Student Doubt Solver Chatbot-ku.*	Super intelligent virtual professor: Enna topic sonnalum instant-aa questions & clear answers explain pannuvaaru.
JWT (python-jose)	JSON Web Token standard for secure, stateless client-server token-based authentication.	*Student/Mentor login pannathum session maintain panna and role-based secure access verify panna.*	College ID Card / Entry Pass: Oru murai pass vaangita, ovvoru murai ID card kaatti ulla pogalam.
Bcrypt	One-way cryptographic password hashing algorithm with salt protection against brute-force.	*Plain text passwords DB-la store aagama, irreversible hashed string-aa encrypt panni save panna.*	Meat Grinder: Paper-ah podda pudichidalam, aana ground meat-la irunthu original paper thirumba edukka mudiyathu.
APScheduler	Advanced in-process background task scheduler for periodic Python execution.	*Every 30 seconds run aagi, quiz deadline mudiyum pothu auto-calculate panni faculty-ku report send panna.*	Automatic Alarm Clock: Time aagiduchu nu alarm adichu email anuppum.
Resend / SMTP	Cloud Transactional Email API service & standard SMTP protocol for automated email dispatch.	*Mentor-ku session summary, weak students alert, and boring class report emails deliver panna.*	Postman: Backend generate panra HTML report-ah faculty mail inbox-ku correct-aa drop pannum.

■ 3. FRONTEND ARCHITECTURE (REACT 19 + VITE + TAILWIND CSS)

Frontend Role: User Interface (UI) and User Experience (UX). Two distinct dashboards exist: **Student Portal** and **Mentor Portal**.

1. React Components & State Management

Tanglish Meaning: *React-la ovvoru screen-um small reusable components-aa pirikapadum. `useState` data store pannum, `useEffect` API call trigger pannum.*

Technical Explanation: React uses Virtual DOM. Instead of re-rendering the whole page, it diffs and updates only changed elements. We use `useState` for timer countdown and answers, `useEffect` for fetching daily topics on load.

Real-life Example: **Quiz countdown timer ovvoru second-um decrement aagum pothu, full page reload aagathu; antha timer number mattum update aagum!**

2. Client-Side Routing (React Router DOM v7)

Tanglish Meaning: *Single Page Application (SPA) koncept. Browser-la page reload aagama URLs switch aagum.*

Technical Explanation: Manages declarative routing with `` and ``. Protected routes inspect the JWT stored in `localStorage` or `sessionStorage` before granting access.

Real-life Example: **User `/mentor/dashboard` click pannal, unauthorized person-ah iruntha login page-ku auto redirect pannidum.**

3. Live Quiz Taking & Anti-Cheat Countdown

Tanglish Meaning: *Frontend timer deadline time-oda match aagi zero aana udane, auto-submit trigger aagum.*

Technical Explanation: The timer calculates `remaining_seconds = deadline_timestamp - current_timestamp`. When it hits 0, `handleSubmitQuiz()` executes automatically.

Real-life Example: **Student submit button click panna maranthalum or tab close panna ninaichalum, time mudinja udane existing answers submit aagidum.**

■■ 4. BACKEND ARCHITECTURE (FASTAPI + ASGI + SQLALCHEMY)

Backend Role: Business logic, database interactions, authentication, scheduled background jobs, and AI API orchestration. Organized cleanly in modular routers: `auth.py`, `students.py`, `timetable.py`, and `chat.py`.

1. FastAPI Request-Response Cycle & Dependency Injection

Tanglish Meaning: *FastAPI Asynchronous Python framework. `async/await` and `Depends(get_db)` use panni DB connection safely manage pandrom.*

Technical Explanation: FastAPI creates lightweight coroutines for incoming requests. `Depends(get_db)` opens a session, injects it into endpoint, and guarantees session close in `finally` block preventing memory leaks.

Real-life Example: **100 students simultaneously quiz submit pannalum, ASGI event loop request-ah queue panni block aagama instantly handle pannum.**

2. Pydantic Schemas vs SQLAlchemy Models (Viva Golden Question!)

Tanglish Meaning: Examiner Question: *'What is the difference between Schema and Model?' — Models create DB tables, Schemas validate API request/response JSON.*

Technical Explanation: SQLAlchemy Model: Defines database structure (`__tablename__`, columns, foreign keys).
Pydantic Schema: Validates HTTP payloads before hitting database (e.g., email format, score integer type).

Real-life Example: **Student score '4/5' string-aa irunthalum percentage 80 integer-aa irukkanumnu check panrathu Pydantic; DB table-la store panrathu SQLAlchemy.**

■ 5. SECURITY & AUTHENTICATION ARCHITECTURE

Security Mechanism	How It Works in EduMentor AI	Why It Is Essential?

Bcrypt Password Hashing	User register/seed aagum pothu password salt add panni hash aagum (`get_password_hash()`). Login-la `verify_password()` match check pannum.	*Database hacker access pannalum original password theriyathu. Plain passwords are NEVER saved in database.*
JWT (JSON Web Token)	Login success aana udane server creates a signed cryptographically tamper-proof token containing `sub` (regno/identity), `role` (student/mentor), and `exp`.	*Stateless: Server database-la session maintain panna theva illa. Token header-la send aagum (`Authorization: Bearer `).*
Role-Based Access Control (RBAC)	`get_current_student` and `get_current_mentor` dependencies token decode panni identity & role verify pannum.	*Student mentor API-ah call panna mudiyathu (`403 Forbidden`). Mentor student quiz attend panna mudiyathu.*
CORS Middleware	FastAPI `CORSMiddleware` configured to allow safe cross-origin communication between Frontend (port 5173) and Backend (port 8000).	*Unauthorized external domains namma backend API-ah misuse pannama protect pannum.*

■ 6. ARTIFICIAL INTELLIGENCE & LLM PIPELINE (GOOGLE GEMINI)

Core AI Engine: Google Gemini API integrated in `backend/app/utils/gemini_service.py`. Dual functionality: (1) Automated Pedagogical Assessment Question Generation, and (2) 24/7 Context-Aware Interactive Doubt Tutor.

1. Dynamic Question Generator with Zero-Downtime Fallback

Tanglish Meaning: *Faculty topic (e.g. 'Deadlocks in OS') enter pannal, Gemini API dynamic MCQ, True/False, Fill in blanks JSON generate pannum.*

Technical Explanation: Prompt engineering instructs Gemini to return strict JSON format with schema `{type, q, a, options}`.
Multi-tier Failover: If Gemini 2.0 Flash is busy, it auto-falls back to `1.5-flash` → `1.5-flash-8b` → Local Curated Topic Question Bank.

Real-life Example: **Internet cut aanaalum or API rate limit (429) vanthaalum system crash aagathu; internal offline Question Bank-la irunthu exam start aagidum!**

2. JSON Sanitization & Parsing Engine (`extract_clean_json`)

Tanglish Meaning: *LLM models silasamayam extra explanations or markdown code blocks (```json ... ```) kudukkum. Athai clean panni pure Python dict-aa convert panrom.*

Technical Explanation: Custom Regex cleaner strips conversational preambles, fences, and removes trailing commas before parsing via `json.loads()`.

Real-life Example: **API response malformed-aa irunthalum server 500 Internal Server Error adikkama smooth-aa recovery pannum.**

3. 24/7 Context-Aware Student Doubt Solving Chatbot

Tanglish Meaning: *Student quiz mudicha piraku or study panrapo varra doubts-ah AI tutor kitta kelvi kettu instant simple explanations edukalam.*

Technical Explanation: Chat messages are saved in database (`ChatMessage` model). The system prompt conditions Gemini to respond as an empathetic academic tutor with examples.

Real-life Example: **Student: 'TCP 3-way handshake enna?' AI: 'Imagine oru phone call: 1. Hello (SYN), 2. Hello kekkuthu (SYN-ACK), 3. Seri pesalam (ACK)!'**

■■ 7. BACKGROUND AUTOMATION & ALERTING ENGINE (`SCHEDULER.PY`)

Why Background Scheduler? Quiz-ku time limit mudiyum pothu human yaarum manual-aa calculate panna koodathu. EduMentor AI-la **APScheduler** background-la run aagi auto-grade panni email report dispatch pannum.

How the 30-Second Polling & 10-Second Deadline Catch Works

Tanglish Meaning: *APScheduler background thread-la every 30 seconds run aagum. Due date & due time compare panni, deadline kitta vantha trigger pannum.*

Technical Explanation: To avoid missing tight deadlines, the scheduler catches a 0-40s window before expiry. A deduplication set (`_sent_reports`) ensures the report is sent exactly ONCE per session.

Real-life Example: **Mentor quiz set pannitu switch off pannitalum, exact-aa deadline mudiyum pothu antha particular mentor mail-ku report poi sernthidum!**

Automated Pedagogical Interventions (What is inside the Email?)

Tanglish Meaning: *Email report faculty-ku comprehensive actionable insights tharum:*

- ***Full Marksheet Table:*** *All students marks + Missed (not attended) list.*
- ***Weak Students List (<50%):*** *Direct names and marks highlighted in RED.*
- ***Not Understood Students:*** *Students who marked 'Puriyala' flag.*
- ***Reteach Suggestion:*** *If weak students > 50%, urgent re-teaching recommendation alert trigger aagum!*

Technical Explanation: Emails are sent via Resend REST API or fallback SMTP to ensure 99.9% inbox delivery.

Real-life Example: **Faculty paper correct panna theva illa; class mudinja 15th minute-la full class diagnosis faculty email inbox-la ready!**

■■ 8. DATABASE ARCHITECTURE – 9 KEY TABLES (`MODELS.PY`)

Table Name	Primary Purpose	Key Columns / Schema	Namma Project-la Ethukku?
students	Student master profile records.	`id, username, regno, dept, year, email, password_hash`	*Student authentication, cohort grouping (1st/2nd/3rd year).*
mentors	Faculty/Mentor master records.	`id, name, dept, identity, password_hash`	*Faculty login and quiz session assignment tracking.*
daily_topics	Active daily topic assigned per cohort.	`id, year, subject, topic, counts, date, due_date, due_time, manual_questions`	*Current live topic & active timer deadline per cohort year.*
topic_history	Immutable append-only history of every assignment.	`id, year, subject, topic, set_by, due_date, due_time, created_at`	*Auditing: Who assigned which topic and when (never overwritten).*
student_scores	Stores assessment results for students.	`id, student_id, topic, score, percentage, date, not_understood`	*Mark records for analytics, leaderboard, weak student detection.*
quiz_attempts	Full question-by-question audit log per attempt.	`id, student_id, topic, answers(JSON), score, percentage`	*Detailed breakdown of student answers vs correct answers.*
boring_reports	Anonymous class engagement & boredom reports.	`id, student_id, topic, date, created_at`	*Tracks students who found class dry/boring for pedagogical review.*
chat_messages	Persistent AI tutor conversation logs.	`id, student_id, sender('student'|'ai'), message, timestamp`	*Stores chat history so student can review past doubts anytime.*
weekly_timetables	Class timetable matrix across the week.	`id, year, day, period, subject, topic`	*Helps mentor schedule topics according to daily class routine.*

■ 9. END-TO-END SYSTEM WORKFLOW (HOW IT WORKS FROM A TO Z)

Step 1: Faculty Sets Daily Topic: Mentor logs in, selects Year (e.g. 2nd Year), enters Subject ('Operating Systems'), Topic ('Page Replacement Algorithms'), specifies Question counts (3 MCQ, 2 TF) and sets a Deadline (e.g. 4:00 PM).

Step 2: AI Question Generation: Backend calls Google Gemini 2.0 Flash with prompt rules. If API works, dynamic questions are returned in JSON; if quota exceeds, curated fallback bank provides instant questions. Topic is activated in DB.

Step 3: Student Enters & Takes Assessment: Students in that year log in. Active quiz banner alerts them with countdown timer. Students answer questions, optionally tick 'I did not understand today\'s session' or report boredom.

Step 4: Immediate Auto-Grading: On submit (or auto-submit when timer expires), backend verifies answers, calculates percentage, saves into `student_scores` and `quiz_attempts`. Student gets immediate scorecard with explanations.

Step 5: Background Engine & Email Report: At 4:00 PM, APScheduler wakes up. It fetches all student scores for that topic. Students who didn't submit are flagged as 'Missed'. Marks table and weak student list are compiled into a responsive HTML email and sent to the mentor!

Step 6: AI Doubt Resolution Support: Struggling students navigate to AI Tutor tab to ask doubts anytime. Gemini AI clarifies complex concepts in conversational, simplified terms.

■ 10. TOP 20 VIVA QUESTIONS & SMART TANGLISH ANSWERS

Q1: Why did you choose FastAPI instead of Django or Flask?

Examiner-ku Answer (English): FastAPI is modern, asynchronous (ASGI), and significantly faster than Flask/Django. It uses Python type hints for automated Pydantic validation and automatically generates interactive Swagger/OpenAPI documentation. Django is heavy with unnecessary built-in template engines, whereas FastAPI is lightweight and ideal for decoupled React frontend SPAs.

Unakku Puriyanum-na (Tanglish Summary): *FastAPI async-aa irunthathaala superfast. Auto documentation (`/docs`) generate aagidum. React kooda REST API connect panna FastAPI is the best modern choice!*

Q2: What is the role of Vite and why not Create-React-App (CRA)?

Examiner-ku Answer (English): Create-React-App is officially deprecated. CRA uses Webpack which bundles the entire code before serving, making builds very slow. Vite uses Native ES Modules (ESM) and esbuild written in Go, offering sub-second dev server start times and instant Hot Module Replacement (HMR).

Unakku Puriyanum-na (Tanglish Summary): *CRA pazhasu, slow-aa irukkum. Vite modern tool, lightning fast-aa compile pannum!*

Q3: How does JWT Authentication work in your project?

Examiner-ku Answer (English): When a student or mentor enters valid credentials, the backend signs a JSON Web Token with a secret key (`SECRET_KEY`), algorithm (`HS256`), and expiration time. The frontend stores this token and sends it in the `Authorization: Bearer ` header for subsequent requests. The backend verifies the token statelessly without needing server-side session lookup.

Unakku Puriyanum-na (Tanglish Summary): *Server token generate panni tharum. Frontend ovvoru request koodavum antha token-ah send panni identity verify pannikkum.*

Q4: What happens if Google Gemini API is down or rate-limited?

Examiner-ku Answer (English): Our system implements a multi-tier fallback mechanism. It first tries `gemini-2.0-flash`, then `gemini-1.5-flash`, and if all cloud LLM calls fail, it automatically falls back to an internal curated question bank (`TOPIC_QUESTION_BANK`) and contextual template generator. The application never crashes.

Unakku Puriyanum-na (Tanglish Summary): *Internet cut aanaalum quota theerthaalum offline question bank-la irunthu test run aagidum, system fail aagathu!*

Q5: How does the timer countdown prevent cheating or delayed submissions?

Examiner-ku Answer (English): The deadline is enforced on both frontend and backend. The frontend runs a JavaScript countdown timer that automatically executes `handleSubmitQuiz()` when it hits zero. Furthermore, the backend stores `due_date` and `due_time`; submissions after deadline cutoff are rejected.

Unakku Puriyanum-na (Tanglish Summary): *Time mudinja udane screen auto submit aagidum, student thappa submit pannaama escape aaga mudiyathu.*

Q6: What is an ORM and why did you use SQLAlchemy?

Examiner-ku Answer (English): Object-Relational Mapping (ORM) maps database tables to Python classes. Instead of writing raw SQL strings (`SELECT * FROM students WHERE id = 1`), we interact with Python objects (`db.query(Student).get(1)`). It provides SQL injection prevention, schema migrations, and database independence.

Unakku Puriyanum-na (Tanglish Summary): *Direct SQL query ezhuthaama Python objects moolama database safe-aa handle panna ORM use pandrom.*

Q7: How do you identify weak students?

Examiner-ku Answer (English): A student is classified as 'Weak' if either: (1) Their assessment score is below 50% (< 50% marks), or (2) The student explicitly checks the 'I did not understand today's session' toggle (`not_understood = True`) during quiz submission.

Unakku Puriyanum-na (Tanglish Summary): *Score 50%-ku keela irunthaalo or 'enakku puriyala' nu click pannalo system avangala weak list-la pottudum.*

Q8: What is APScheduler and why is it needed?

Examiner-ku Answer (English): Advanced Python Scheduler (APScheduler) runs in-process background tasks. It polls every 30 seconds to monitor active deadlines. In the last 10-40 seconds of a quiz window, it compiles session attendance, identifies weak students, and automatically triggers email delivery without human intervention.

Unakku Puriyanum-na (Tanglish Summary): *Background-la running alarm clock maathiri; deadline mudiyum pothu automatic-aa calculate panni faculty-ku email anuppum.*

Q9: How are passwords stored securely?

Examiner-ku Answer (English): Passwords are NEVER stored as plain text. We use the `bcrypt` cryptographic hashing library with random salting. Even if someone obtains full database access, bcrypt is computationally expensive and mathematically infeasible to reverse engineer.

Unakku Puriyanum-na (Tanglish Summary): *Bcrypt one-way hashing use pandrom. Password hash aagi irukkum, yarukkum original password theriyathu.*

Q10: What is the difference between DailyTopic and TopicHistory tables?

Examiner-ku Answer (English): `DailyTopic` holds only the single currently active topic for each year cohort (it is updated/upserted). `TopicHistory` is an append-only immutable audit trail that saves every topic assigned historically along with the mentor's name and timestamps.

Unakku Puriyanum-na (Tanglish Summary): *DailyTopic innaiki active-aa irukra topic-ah mattum store pannum. TopicHistory pazhaya ella topics records-um audit trail-aa save pannum.*

Q11: How do you handle Cross-Origin Resource Sharing (CORS)?

Examiner-ku Answer (English): Since the React frontend runs on port 5173 and FastAPI runs on port 8000, browsers block cross-origin HTTP requests by default. We configure FastAPI's `CORSMiddleware` with allowed origins, methods, and headers to enable safe cross-origin communication.

Unakku Puriyanum-na (Tanglish Summary): *Frontend and Backend different ports-la run aagurathaala browser security block pannaama irukka CORS configure pandrom.*

Q12: How does the AI Chatbot retain context?

Examiner-ku Answer (English): Chat history is persisted in the `chat_messages` table with student ID, sender role (`student` or `ai`), and timestamp. When a query is made, relevant context from previous interactions can be injected into the prompt, and past doubts remain accessible across logins.

Unakku Puriyanum-na (Tanglish Summary): *Chat messages DB-la store aavathaala student logout panni thirumba vanthaalum pazhaya doubts review pannalam.*

Q13: Why Tailwind CSS v4 over traditional CSS files?

Examiner-ku Answer (English): Tailwind CSS uses utility classes directly in JSX markup. It eliminates CSS file bloat, provides a standardized spacing/typography design system, enables lightning-fast prototyping, and includes built-in dark mode and responsive breakpoints (`md:`, `lg:`).

Unakku Puriyanum-na (Tanglish Summary): *Custom CSS ezhuthi waste panna theva illa. Class names vachae responsive and modern glassmorphic look create pannalam.*

Q14: What is the purpose of Pydantic's `BaseModel`?

Examiner-ku Answer (English): Pydantic's `BaseModel` is used in `schemas.py` to define data contracts. It automatically validates incoming JSON types, rejects extra/invalid fields, and serializes Python data objects into clean JSON API responses.

Unakku Puriyanum-na (Tanglish Summary): *API request and response data types-ah strict-aa check panni validate panra security layer.*

Q15: How does the email alerting system work without blocking user requests?

Examiner-ku Answer (English): Email dispatch is handled either through background scheduler jobs or asynchronous execution using Resend API / SMTP. The primary user flow doesn't hang waiting for third-party email delivery.

Unakku Puriyanum-na (Tanglish Summary): *Email anuppurathaala student app hang aagathu; background worker process anuppidum.*

Q16: What is a Single Page Application (SPA)?

Examiner-ku Answer (English): An SPA loads a single HTML shell (`index.html`) on initial load. Subsequent page transitions occur dynamically via JavaScript DOM manipulation without full browser page reloads, providing a desktop-app-like experience.

Unakku Puriyanum-na (Tanglish Summary): *Page refresh aagama content mattum fast-aa maarum app structure.*

Q17: What happens when a student marks a class as 'Boring'?

Examiner-ku Answer (English): The system records the response in the `boring_reports` table linked to the student ID, topic, and date. This qualitative metric is tracked to help mentors assess engagement and adapt pedagogical techniques.

Unakku Puriyanum-na (Tanglish Summary): *Class dull-aa iruntha anonymous-aa student report pannalam. Faculty teaching style refine panna useful data.*

Q18: What is the database auto-seeder (`seed_database`)?

Examiner-ku Answer (English): On server startup, `main.py` runs `seed_database()` which checks if initial mock student accounts, mentor credentials, and timetables exist. If empty, it pre-populates the database so the system is immediately testable and functional.

Unakku Puriyanum-na (Tanglish Summary): *Server start aagum pothu sample students and mentors data illana automatic-aa create panni ready pannidum.*

Q19: What is ASGI and how does Uvicorn fit in?

Examiner-ku Answer (English): ASGI (Asynchronous Server Gateway Interface) is the asynchronous successor to WSGI. It allows Python web applications to handle asynchronous protocols like WebSockets, HTTP/2, and long-polling. Uvicorn is the high-performance ASGI web server hosting FastAPI.

Unakku Puriyanum-na (Tanglish Summary): *WSGI synchronous pazhaya model. ASGI async modern model. Uvicorn is the server that runs FastAPI on ASGI.*

Q20: What are the future enhancements of EduMentor AI?

Examiner-ku Answer (English): Future roadmap includes: (1) Voice-based oral viva testing via Whisper AI, (2) Adaptive difficulty (questions automatically become harder or easier based on real-time performance), (3) Mobile application integration via React Native, and (4) Predictive analytics predicting university semester exam grades.

Unakku Puriyanum-na (Tanglish Summary): *Voice based viva, dynamic difficulty adjustment, mobile app and semester score prediction add pannalam!*

■ 11. VIVA HALL GOLDEN RULES (CONFIDENCE CHECKLIST)

1. Don't Say 'I Don't Know': Examiner kelvi keta udane blank aagathinga. Say: 'Sir, in our current architecture we handled this using X, but we can also extend it using Y.'

2. Explain the Problem First: 'Before our project, faculty had to wait 3 months for semester results. With EduMentor AI, they get daily feedback in 15 minutes.'

3. Highlight the Fallback AI: Examiner will definitely ask 'Gemini limit mudinja enna pannuvinga?' Answer: 'We implemented a multi-tier fallback with an offline curated question bank so zero downtime!'

4. Show Ownership: Confidently name the files: 'Frontend is in React 19 (`StudentDashboardPage.jsx`), Backend is in FastAPI (`students.py`), Database is managed by SQLAlchemy (`models.py`) and Background tasks by APScheduler (`scheduler.py`).'